Swinburne University of Technology

Faculty of Science, Engineering and Technology

ASSIGNMENT COVER SHEET

Subject Code: COS30008
Subject Title: Data Structures and Patterns
Assignment number and title: 2, Indexers, Method Overriding, and Lambdas
Due date: March 1st, 2026
Lecturer: Dr. Bui Van Vuong

Your name: Nguyen Duc Manh **Your student id:** 105547489

Check Tutorial	Mon 10:30	Mon 14:30	Tues 08:30	Tues 10:30	Tues 12:30	Tues 14:30	Tues 16:30	Wed 08:30	Wed 10:30	Wed 12:30	Wed 14:30

Marker's comments:

Problem	Marks	Obtained
1	48	
2	30+10= 40	
3	58	
Total	146	

Extension certification:

This assignment has been given an extension and is now due on _____

Signature of Convener:_____

```cpp
// 28/01/2026

#include <iostream>
#include "IntVector.h"
#include <stdexcept>

using namespace std;

IntVector::IntVector(const int aArrayOfIntegers[], size_t aNumberOfElements) {
    fNumberOfElements = aNumberOfElements;
    fElements = new int[fNumberOfElements]; // First initiate the fElements
    // Copying each ele from the og to this class
    for (size_t i = 0; i < fNumberOfElements; i++) {
        fElements[i] = aArrayOfIntegers[i];
    }
}

IntVector::~IntVector() {
    delete[] fElements;
};

size_t IntVector::size() const { return fNumberOfElements; };

void IntVector::swap(size_t aSourceIndex, size_t aTargetIndex) {
    if (aSourceIndex < fNumberOfElements && aTargetIndex < fNumberOfElements) {
        int temp;
        temp = fElements[aSourceIndex];
        fElements[aSourceIndex] = fElements[aTargetIndex];
        fElements[aTargetIndex] = temp;
    }
    else {
        throw out_of_range("Illegal vector indices");
    }
};

const int IntVector::get(size_t aIndex) const {
    return (*this)[aIndex];
};

const int IntVector::operator[](size_t aIndex) const {
    if (aIndex < fNumberOfElements) {
        return fElements[aIndex];
    }
    else {
        throw out_of_range("Illegal vector index");
    }
};
```

```cpp
#include "SortableIntVector.h"

SortableIntVector::SortableIntVector(const int aArrayOfIntegers[], size_t aNumberOfElements)
    : IntVector(aArrayOfIntegers, aNumberOfElements)
{ };

void SortableIntVector::sort(Comparable aOrderFunction) {
    size_t ArrSize = this->size();
    for (int i = 0; i < ArrSize; i++) {
        for (size_t j = 0; j < ArrSize - 1 - i; j++) {
            // since the larger ele is sorted, we don't need to check for it anymore
            size_t leftVal = this->get(j);
            size_t rightVal = this->get(j + 1);
            if (!aOrderFunction(leftVal, rightVal)) {
                this->swap(j, j+1);
            }
        }
    }
};
```

```cpp
#include "ShakerSortableIntVector.h"

ShakerSortableIntVector::ShakerSortableIntVector(const int aArrayOfIntegers [], size_t aNumberOfElements)
    : SortableIntVector(aArrayOfIntegers, aNumberOfElements) { };

void ShakerSortableIntVector::sort(Comparable aOrderFunction)
{
    int arrLength = this->size();
    for (int i = 0; i < arrLength/2; i++) {
        for (int j = 0; j < arrLength - 1 - i; j++) {
            int leftVal = this->get(j);
            int rightVal = this->get(j + 1);
            if (aOrderFunction(leftVal, rightVal)) {
                this->swap(j, j+1);
            }
        }
        // now going backward
        for (int k = arrLength - 2 - i; k >= i; k--) {
            int leftVal = this->get(k);
            int rightVal = this->get(k + 1);
            if (aOrderFunction(leftVal, rightVal)) {
                this->swap(k, k + 1);
            }
        }
    }
};
```

```cpp

// Problem Set 2, 2022

#include <iostream>
#include <stdexcept>

using namespace std;

//#define P1
//#define P2
#define P3

#ifdef P1

#include "IntVector.h"

void runP1()
{
    int lArray[] = { 34, 65, 890, 86, 16, 218, 20, 49, 2, 29 };
    size_t lArrayLength = sizeof(lArray) / sizeof(int);

    IntVector lVector( lArray, lArrayLength );

    cout << "Test range check:" << endl;

    try
    {
        int lValue = lVector[lArrayLength];

        cerr << "Error, you should not see " << lValue << " here!" << endl;
    }
    catch (out_of_range e)
    {
        cerr << "Properly caught error: " << e.what() << endl;
    }
    catch (...)
    {
        cerr << "This message must not be printed!" << endl;
    }

    cout << "Test swap:" << endl;

    try
    {
        cout << "lVector[3] = " << lVector[3] << endl;
        cout << "lVector[6] = " << lVector[6] << endl;

        lVector.swap( 3, 6 );
```

```cpp

        cout << "lVector.get( 3 ) = " << lVector.get( 3 ) << endl;
        cout << "lVector.get( 6 ) = " << lVector.get( 6 ) << endl;

        lVector.swap( 5, 20 );

        cerr << "Error, you should not see this message!" << endl;
    }
    catch (out_of_range e)
    {
        cerr << "Properly caught error: " << e.what() << endl;
    }
    catch (...)
    {
        cerr << "Error, this message must not be printed!" << endl;
    }
}

#endif

#ifdef P2

#include "SortableIntVector.h"

void runP2()
{
    int lArray[] = { 34, 65, 890, 86, 16, 218, 20, 49, 2, 29 };
    size_t lArrayLength = sizeof(lArray) / sizeof(int);

    SortableIntVector lVector( lArray, lArrayLength );

    cout << "Bubble Sort:" << endl;

    cout << "Before sorting:" << endl;

    for ( size_t i = 0; i < lVector.size(); i++ )
    {
        cout << lVector[i] << ' ';
    }

    cout << endl;

    // Use a lambda expression here that orders integers in increasing order.
    // The lambda expression does not capture any variables of throws any exceptions.
    // It has to return a bool value.
    lVector.sort(
        [](int leftVal, int rightVal) -> bool {
```

```
            return leftVal <= rightVal;
        }
    );

    cout << "After sorting:" << endl;

    for ( size_t i = 0; i < lVector.size(); i++ )
    {
        cout << lVector[i] << ' ';
    }

    cout << endl;
}

#endif

#ifdef P3

#include "ShakerSortableIntVector.h"

void runP3()
{
    int lArray[] = { 34, 65, 890, 86, 16, 218, 20, 49, 2, 29 };
    size_t lArrayLength = sizeof(lArray) / sizeof(int);

    ShakerSortableIntVector lVector( lArray, lArrayLength );

    cout << "Cocktail Shaker Sort:" << endl;

    cout << "Before sorting:" << endl;

    for ( size_t i = 0; i < lVector.size(); i++ )
    {
        cout << lVector[i] << ' ';
    }

    cout << endl;

    // sort in decreasing order
    lVector.sort();

    cout << "After sorting:" << endl;

    for ( size_t i = 0; i < lVector.size(); i++ )
    {
        cout << lVector[i] << ' ';
    }

    cout << endl;
```

```
}

#endif

int main()
{
#ifdef P1

    runP1();

#endif

#ifdef P2

    runP2();

#endif

#ifdef P3

    runP3();

#endif

    return 0;
}

```